

Database Systems (Model Solution)

Question 01: Answer the following questions:

I. Discuss the difference between DBMS, and Flat File system

- A **Flat File System** stores data in simple, unstructured text files (e.g., CSV, TXT). Each file typically contains all the data for a single application, with no relationships between files. Data management is done manually by application programs.
- A **Database Management System (DBMS)** is specialized software that provides a structured way to store, retrieve, and manage data. It uses tables, enforces relationships, and ensures data integrity through constraints.

Feature	Flat File System	DBMS
Data Structure	No structure; data stored in plain files	Structured into tables with rows & columns
Redundancy	High redundancy (same data repeated)	Minimal redundancy (normalization)
Data Integrity	No enforcement; errors common	Enforced via constraints (PK, FK, checks)
Data Sharing	Difficult; each app has its own files	Multiple users/apps share same database
Concurrent Access	Not supported; leads to conflicts	Handles concurrency with locking/transactions
Security	Minimal (file permissions only)	User authentication, access control
Query Capability	No query language; data read manually	Powerful query languages (SQL)
Backup/Recovery	Manual, no automatic recovery	Automated backup and crash recovery

II. Explain the differences between Two-Tier and Three-Tier client-server architectures, and provide one relevant use case for each.

Two-Tier Architecture

In a two-tier architecture, the system is divided into two layers:

1. **Client Tier (Presentation Layer):** Handles the user interface and sometimes part of the application logic.
2. **Server Tier (Data Layer):** Manages the database and data processing.

The client communicates directly with the database server using queries (e.g., SQL). Application logic may be split between the client and the server (often via stored procedures).

Three-Tier Architecture

In a three-tier architecture, the system is divided into three logical layers:

1. **Presentation Tier (Client):** The user interface (web browser, mobile app, or desktop GUI).
2. **Application Tier (Middle Tier/Business Logic Layer):** Contains the business rules, processes requests, and acts as an intermediary between the client and the database.
3. **Data Tier (Database Server):** Stores and manages data.

The client never interacts directly with the database; all requests go through the application server, which validates data, applies business logic, and then queries the database.

III. Identify three examples, with proper justification, where the use of a Database Management System (DBMS) would not be appropriate.

1. A **smart thermostat** that stores temperature settings and schedules locally.
2. **Personal Recipe Collection** (few recipes, search by name/ingredients, local file)
3. A **personal to-do list application** used by one person on their laptop.

Question 02: write the SQL queries for the given questions below

- 1. Display all players in such a way that players who debuted earlier appear first.**

```
SELECT *  
FROM PLAYERS  
ORDER BY YEAR_OF_DEBUT ASC;
```

- 2. List all players who started playing international cricket after 2014 but not later than 2020, and whose last name doesn't begin with the letter 'K'.**

```
SELECT *  
FROM PLAYERS  
WHERE YEAR_OF_DEBUT BETWEEN 2015 AND 2020  
AND LAST_NAME NOT LIKE 'K%';
```

- 3. List all players who managed to score 50 or more runs in at least one of their matches, also display the match's name in which they achieved this milestone.**

```
SELECT P.PLAYER_ID, P.FIRST_NAME, P.LAST_NAME, M.MATCH_NAME  
FROM PLAYERS P, STATISTICS S, MATCHES M  
WHERE P.PLAYER_ID = S.PLAYER_ID  
AND S.MATCH_ID = M.MATCH_ID  
AND S.RUNS >= 50;
```

- 4. List all players whose first names include the letter 'e' anywhere and who made their international debut in 2018 or any later year.**

```
SELECT *  
FROM PLAYERS  
WHERE FIRST_NAME LIKE '%e%'  
AND YEAR_OF_DEBUT >= 2018;
```

- 5. List all coaches along with their teams, sorted by coach's last name in ascending order, and for same coach, teams with more points first.**

```
SELECT C.FIRST_NAME, C.LAST_NAME, T.TEAM_NAME, PT.POINTS  
FROM COACHES C, TEAMS T, POINTS_TABLE PT  
WHERE C.COACH_ID = T.COACH_ID  
AND T.TEAM_ID = PT.TEAM_ID  
ORDER BY C.LAST_NAME ASC, PT.POINTS DESC;
```

Question 03: Using the Database schema of Question 2, you need to write the DDL statement while considering the below requirements:

1. TEAMS table

```
CREATE TABLE TEAMS (  
    TEAM_ID      INT    NOT NULL    PRIMARY KEY,  
    TEAM_NAME    VARCHAR(100) NOT NULL    UNIQUE,  
    COACH_ID     INT ,  
    FOREIGN KEY (COACH_ID) REFERENCES COACHES(COACH_ID)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);
```

2. PLAYERS table

```
CREATE TABLE PLAYERS (  
    PLAYER_ID     INT    NOT NULL    PRIMARY KEY,  
    FIRST_NAME    VARCHAR(50) NOT NULL,  
    LAST_NAME     VARCHAR(50) NOT NULL,  
    TEAM_ID       INT          NOT NULL,  
    YEAR_OF_DEBUT INT          NOT NULL,  
  
    FOREIGN KEY (TEAM_ID) REFERENCES TEAMS(TEAM_ID)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE,  
    CONSTRAINT CHK_DEBUT_YEAR CHECK (YEAR_OF_DEBUT >= 2010)  
);
```

3. COACHES table

```
CREATE TABLE COACHES (  
    COACH_ID      INT          PRIMARY KEY,  
    FIRST_NAME    VARCHAR(50)  NOT NULL,  
    LAST_NAME     VARCHAR(50)  NOT NULL,  
    SPECIALIZATION VARCHAR(100) NOT NULL,  
    EMAIL         VARCHAR(100) NOT NULL    UNIQUE  
);
```

4. VENUES table

```
CREATE TABLE VENUES (  
    VENUE_ID      INT    PRIMARY KEY,  
    VENUE_NAME    VARCHAR(100) NOT NULL,  
    CITY          VARCHAR(50)  NOT NULL,  
    COUNTRY       VARCHAR(50)  NOT NULL  
);
```

5. MATCHES table

```
CREATE TABLE MATCHES (  
    MATCH_ID      INT    PRIMARY KEY,  
    MATCH_NAME    VARCHAR(200) NOT NULL,  
    VENUE_ID      INT          NOT NULL,  
    DATE          DATE          NOT NULL,
```

```

FOREIGN KEY (VENUE_ID) REFERENCES VENUES(VENUE_ID)
ON DELETE CASCADE
ON UPDATE CASCADE
);

```

6. STATISTICS table

```

CREATE TABLE STATISTICS (
    STAT_ID      INT          PRIMARY KEY,
    PLAYER_ID    INT          NOT NULL,
    MATCH_ID     INT          NOT NULL,
    RUNS         INT          DEFAULT 0,
    WICKETS      INT          DEFAULT 0,
    RATING       DECIMAL(3,2),

    CONSTRAINT CHK_RUNS      CHECK (RUNS >= 0),
    CONSTRAINT CHK_WICKETS   CHECK (WICKETS >= 0),
    CONSTRAINT CHK_RATING    CHECK (RATING BETWEEN 0 AND 10),

    FOREIGN KEY (PLAYER_ID) REFERENCES PLAYERS(PLAYER_ID)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
    FOREIGN KEY (MATCH_ID) REFERENCES MATCHES(MATCH_ID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

```

7. POINTS TABLE table

```

CREATE TABLE POINTS_TABLE (
    TEAM_ID      INT          PRIMARY KEY,
    POINTS       INT          NOT NULL  DEFAULT 0,
    LAST_UPDATED DATE        NOT NULL  DEFAULT CURRENT_DATE,

    FOREIGN KEY (TEAM_ID) REFERENCES TEAMS(TEAM_ID)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
    CONSTRAINT CHK_POINTS   CHECK (POINTS >= 0)
);

```